Ticket Support System

Copy

Laravel Project-Based Test - Intermediate Level

Getting Started

This is a Laravel intermediate-level test project to manage support tickets between customers, agents, supervisors, and administrators.

The Goal

Of course this has a goal of something.

The goal is to assess how well you can build a Laravel application that is not only working, but also maintainable, secure, testable, and closer to a real-world production system.

This test will cover:

- MVC
- Authentication
- Authorization
- Role-based access control
- CRUD and Resource Controllers
- Eloquent relationships
- Database migrations and seeders
- Factories
- Form validation using Request classes
- File management
- Polymorphic relationships
- Pagination
- Filtering, searching, and sorting

- Notifications
- Queues and jobs
- REST API using Laravel Sanctum
- Activity logging
- Basic reporting and export
- Testing with Pest or PHPUnit
- Basic frontend implementation using starter kits, templates, or custom UI
- Third-party integration
- Not losing your soul while building another CRUD app

The expected level is **Intermediate**.

This means the project should not only work. The structure should also make sense. Controllers should not become a landfill. Business logic should not be dumped randomly like confetti at 2 AM.

The Task

Build a support ticket management system where customers can create tickets, agents can handle assigned tickets, supervisors can monitor team progress, and administrators can manage the entire system.

Customers register as users and create tickets. Admins or supervisors assign tickets to agents. Agents work on tickets, update statuses, add comments or internal notes, and resolve issues. Customers can follow the progress and respond through comments.

This challenge gives developers freedom to choose packages, frontend approach, UI design, and technical structure. The goal is to prove that the system can deliver results without becoming spaghetti with login buttons.

User Roles

Every user must have one of these roles:

1. Customer

This is the default role after registration.

Customers can:

- Register and log in
- Create new tickets
- View only tickets created by themselves
- Add public comments to their own tickets
- Upload attachments to their own tickets
- View ticket details
- View simplified ticket activity history
- Reopen a resolved or closed ticket if allowed by workflow rules

Customers cannot:

- Edit tickets after submission
- Delete tickets
- Assign agents
- View internal notes
- View other customers' tickets
- Change ticket status directly, except reopening allowed tickets

2. Agent

Agents handle tickets assigned to them.

Agents can:

- View tickets assigned to them
- Update ticket status based on allowed workflow rules
- Add public comments
- Add internal notes
- Upload attachments
- Mark tickets as resolved

- View activity history for assigned tickets

Agents cannot:

- Delete tickets
- Assign tickets to themselves
- Manage users
- Manage master data
- Access tickets assigned to other agents unless allowed by supervisor or admin

3. Supervisor

Supervisors monitor ticket progress and agent workload.

Supervisors can:

- View tickets assigned to agents under their team
- Reassign tickets between agents
- View dashboard reports
- View overdue tickets
- View escalated tickets
- Add public comments
- Add internal notes
- Export ticket reports
- View logs for tickets under their supervision

Supervisors cannot:

- Manage system-wide settings
- Delete users
- Manage global roles
- Delete tickets unless explicitly implemented as a bonus feature

4. Administrator

Administrators manage the whole system.

Admins can:

- View all tickets
- Create, edit, and manage tickets
- Assign tickets to agents
- Reassign tickets
- Manage users
- Manage roles
- Manage categories
- Manage labels
- Manage priorities
- Manage SLA rules
- View all activity logs
- View system dashboard
- Configure ticket workflow settings if implemented

Authentication

There should be login and register functionality.

You may use one of these:

- Laravel Breeze
- Laravel Jetstream
- Laravel Fortify
- Laravel UI
- Custom authentication implementation

New registered users must automatically receive the **Customer** role.

The system must seed these users:

```
Admin
Email: admin@admin.com
Password: password

Supervisor
Email: supervisor@admin.com
Password: password

Agent
Email: agent@admin.com
Password: password

Customer
Email: customer@demo.com
Password: password
```

You may add more seeded users if needed for testing and demo data.

Core Ticket Features

Each ticket must have:

Field	Requirement
Ticket number	Auto-generated, unique, human-readable
Title	Required
Description	Required
Priority	Required
Status	Required
Category	Required
Labels	Optional, multiple
Created by	Customer user
Assigned agent	Nullable, only admin or supervisor can assign
Due date	Generated from SLA rules
Resolved at	Nullable
Closed at	Nullable
Attachments	Optional, multiple
Comments	Multiple
Internal notes	Multiple, visible only to agent, supervisor, and admin

Example ticket number:

```
TCK-2026-000001
```

Ticket number generation must be handled consistently. Do not let users manually type ticket numbers. That is how chaos gets a database column.

Ticket Status Lifecycle

The ticket must follow a controlled workflow.

Allowed statuses:

```
Open
Assigned
In Progress
Waiting for Customer
Resolved
Closed
Reopened
Escalated
```

Status Transition Rules

The system must prevent invalid status transitions.

Example rules:

From	Allowed Next Status
Open	Assigned, Closed
Assigned	In Progress, Escalated
In Progress	Waiting for Customer, Resolved, Escalated
Waiting for Customer	In Progress, Resolved
Resolved	Closed, Reopened
Closed	Reopened
Reopened	Assigned, In Progress
Escalated	In Progress, Resolved

This logic should not be random `if else` soup inside the controller.

Put the logic somewhere sane, such as:

```
app/Services/TicketStatusService.php
```

Or use enums, state pattern, or another clean approach.

The important thing is this: status rules must be centralized, reusable, and testable.

SLA Rules

Add SLA rules based on ticket priority.

Example:

Priority	Response Time	Resolution Time
Low	24 hours	5 days
Medium	8 hours	3 days
High	4 hours	1 day
Critical	1 hour	8 hours

The system must:

- Automatically calculate ticket due date
- Mark tickets as overdue when resolution time is exceeded
- Show overdue tickets in dashboard
- Allow admins to manage SLA rules

Bonus points if SLA calculations ignore weekends or holidays.

That kind of business logic is cursed, but it is also very real.

Ticket Comments and Internal Notes

Tickets must support two types of messages.

Public Comments

Visible to:

- Customer
- Assigned agent

- Supervisor
- Admin

Public comments are used for normal communication between customer and support team.

Internal Notes

Visible only to:

- Agent
- Supervisor
- Admin

Customers must never see internal notes.

Not in the UI.

Not in the API.

Not by inspecting the page.

Not by poking around like a goblin in DevTools.

Attachments

The system must support multiple file attachments.

Requirements:

- Use polymorphic relationship
- Store files in `storage/app/public`
- Limit file size to 2 MB
- Allow only safe file types:

```
jpg, jpeg, png, pdf, doc, docx, xls, xlsx
```

- Store original filename
- Store generated filename

- Store file size
- Store MIME type
- Store uploader ID
- Prevent users from accessing attachments from tickets they are not allowed to view

Suggested table:

```
attachments
- id
- attachable_type
- attachable_id
- uploaded_by
- original_name
- stored_name
- path
- mime_type
- size
- created_at
- updated_at
```

Attachments may belong to tickets or comments.

Example relationships:

```
Ticket morphMany Attachment
Comment morphMany Attachment
Attachment morphTo attachable
```

Dashboard Requirements

Admin Dashboard

Must show:

- Total tickets
- Tickets by status
- Tickets by priority
- Tickets by category
- Overdue tickets

- Unassigned tickets
- Tickets created this week
- Average resolution time
- Top 5 agents by resolved tickets

Supervisor Dashboard

Must show:

- Tickets assigned to agents under their team
- Open tickets
- Overdue tickets
- Escalated tickets
- Agent workload
- Average resolution time per agent

Agent Dashboard

Must show:

- My assigned tickets
- My overdue tickets
- My tickets by status
- Recently updated tickets

Customer Dashboard

Must show:

- My tickets
- Open tickets
- Resolved tickets
- Recently updated tickets

Filtering, Searching, Sorting, and Pagination

Ticket lists must support filters.

Filters

- Status
- Priority
- Category
- Label
- Assigned agent
- Created date range
- Due date range
- Overdue only

Search

Search by:

- Ticket number
- Title
- Description
- Customer name
- Customer email

Sorting

Sort by:

- Created date
- Updated date
- Priority

- Due date
- Status

Pagination is required.

Default pagination:

```
10 entries per page
```

Filtering and searching must preserve pagination state where possible.

Authorization

Use Laravel authorization properly.

Required:

- Policies
- Gates where appropriate
- Middleware where appropriate

Do not only hide buttons in Blade, React, Livewire, or any frontend layer and call it security.

That is not security.

That is UI cosplay.

Examples of expected policies:

```
TicketPolicy
CategoryPolicy
UserPolicy
AttachmentPolicy
CommentPolicy
SlaRulePolicy
```

Authorization must be enforced on the backend.

Every protected action must check permission properly.

Notifications

The system must send notifications for important ticket events.

Required notifications:

Event	Recipient
Ticket created	Admin
Ticket assigned	Assigned agent
Ticket commented	Related users
Ticket resolved	Customer
Ticket escalated	Supervisor/Admin
SLA overdue	Supervisor/Admin

Use Laravel Notifications.

At least one notification must be queued.

Mail can be tested using:

- Mailtrap
- Mailpit
- Log mail driver
- Gmail SMTP

Do not hardcode real email credentials in the repository. That is not a feature. That is a future incident report.

Queues and Jobs

Use Laravel queues for at least one background process.

Examples:

- Sending email notifications

- Checking overdue tickets
- Processing uploaded files
- Exporting reports

Recommended command:

```
php artisan queue:work
```

Add clear queue setup instructions in the README.

Bonus if the project includes a scheduled command for checking overdue tickets.

Example:

```
php artisan tickets:check-overdue
```

Activity Logs

Track important events.

Logs must include:

- Who performed the action
- What action happened
- Which ticket was affected
- Old value
- New value
- Timestamp

Example events:

```
Ticket created
Ticket assigned
Priority changed
Status changed
Comment added
Attachment uploaded
Ticket resolved
Ticket reopened
Ticket escalated
SLA overdue detected
```

Admins must have a `Logs` page.

Supervisors may view logs only for tickets under their team.

Customers may view simplified logs for their own tickets.

You may implement this manually or use a package such as:

```
spatie/laravel-activitylog
```

Admin CRUD Modules

Admins must be able to manage:

- Users
- Categories
- Labels
- Priorities
- SLA Rules

Each CRUD must include:

- List page
- Create form
- Edit form
- Delete action
- Validation
- Pagination
- Search where relevant

Soft delete is recommended for important master data.

Do not permanently delete important records if they are already connected to tickets. That is how history becomes soup.

Database Requirements

Use proper relationships.

Expected models:

```
User
Role
Team
Ticket
Category
Label
Priority
Comment
Attachment
ActivityLog
SlaRule
```

Expected relationships:

```
User hasMany Tickets as creator
User hasMany Tickets as assigned agent
Ticket belongsTo User as creator
Ticket belongsTo User as assignedAgent
Ticket belongsTo Category
Ticket belongsTo Priority
Ticket belongsToMany Label
Ticket hasMany Comment
Ticket morphMany Attachment
Comment morphMany Attachment
Ticket hasMany ActivityLog
Team hasMany Users
Supervisor hasMany Agents
```

Use:

- Migrations
- Seeders
- Factories

- Proper indexes
- Foreign key constraints

Add indexes for frequently filtered columns:

```
status
priority_id
category_id
assigned_agent_id
created_by
due_at
created_at
updated_at
```

Validation

Use Form Request classes.

Required examples:

```
StoreTicketRequest
UpdateTicketRequest
StoreCommentRequest
StoreInternalNoteRequest
UpdateTicketStatusRequest
StoreAttachmentRequest
StoreUserRequest
UpdateUserRequest
StoreSlaRuleRequest
```

Validation must include:

- Required fields
- File type restrictions
- File size restrictions
- Valid enum or status values
- Role-based validation rules
- Safe input handling

Example:

Only admin or supervisor can send `assigned_agent_id` when updating a ticket.

If a customer tries to send `assigned_agent_id`, the backend should ignore or reject it. Prefer rejecting it with clear validation or authorization handling.

Frontend Requirement

You may use:

- Blade
- Livewire
- Inertia + React
- Filament
- Laravel starter kits
- Any Laravel-friendly admin template
- Custom design with Tailwind CSS

The UI must include:

- Sidebar navigation
- Topbar or user menu
- Dashboard cards
- Data tables
- Filters
- Search
- Pagination
- Form validation errors
- Status badges
- Priority badges
- Empty states
- Confirmation modal before destructive actions
- Responsive layout for important pages

> Bonus if the UI does not look like it was assembled during a power outage.

Frontend Wireframe Suggestion

You do not need to follow this exactly, but the final UI should cover the same functional ideas.

Use this section as frontend direction. Candidates may also create the wireframe using Excalidraw, Figma, Penpot, Whimsical, or any similar tool.

1. Role-Aware App Shell

Purpose: provide a consistent layout for all roles.

> ⓘ Sidebar visibility must depend on role.

Example:

Role	Visible Menu
Customer	Dashboard, Tickets
Agent	Dashboard, Assigned Tickets
Supervisor	Dashboard, Team Tickets, Reports
Admin	Dashboard, Tickets, Users, Master Data, SLA Rules, Logs

2. Customer Ticket List

Purpose: let customers view and track their own tickets.

> ⓘ Important UI notes:
>
> - Customer only sees their own tickets
> - Ticket title should be clickable
> - Empty state should be friendly
> - Add a clear `New Ticket` button

Chaos-friendly empty state example:

```
No tickets yet. Either everything works, or the bugs are hiding.
```

3. Create Ticket Page

Purpose: allow customer to submit a new ticket.

> Attachment rules should be visible to the user.

Example:

```
Allowed files: jpg, jpeg, png, pdf, doc, docx, xls, xlsx. Max 2 MB each.
```

---

4. Ticket Detail Page

Purpose: show full ticket context, communication, and history.

> - Internal notes must not be rendered for customers.
> - Not hidden with CSS.
> - Not sent through API.
> - Not leaked in JSON.
> - Gone. Vanished. Spiritual firewall.

5. Agent or Supervisor Workboard

Purpose: help support staff manage active tickets.

Each card should show:

- Ticket number
- Title
- Priority
- Due date
- Customer name
- Assigned agent

Overdue critical ticket UI idea:

```
This ticket is on fire.
```

Use this carefully. Funny, but still professional enough for humans wearing shoes.

6. Admin Dashboard

Purpose: give admins a system-wide overview.

- Dashboard should use real data.
- Do not hardcode dashboard numbers unless clearly marked as seed/demo data.
- Dashboard cosplay is still cosplay.

7. Assignment Modal

Purpose: allow admin or supervisor to assign tickets cleanly.

Only admin and supervisor should access this action.

8. Mobile-Friendly Ticket Detail

Purpose: make sure important pages do not break on smaller screens.

Export Feature

Supervisors and admins must be able to export ticket reports.

Export format:

```
CSV
```

Export filters:

- Date range
- Status
- Priority
- Agent
- Category

The export can be synchronous for small data.

Queued export gets bonus points.

Security Requirements

The project must handle these properly:

- Authorization on every protected action
- File access permission
- CSRF protection for web routes
- Rate limiting for API routes
- Validation for uploaded files
- No exposed sensitive `.env` values
- No hardcoded credentials except seed demo users
- Prevent mass assignment issues
- Use `$fillable` or `$guarded` properly
- Do not expose internal notes to customers
- Do not expose unauthorized attachments

- Do not trust frontend-only checks

Security is not hiding a button and praying.

Testing Requirement

Use Pest or PHPUnit.

Minimum required tests:

Feature Tests

- Customer can register and login
- Customer can create ticket
- Customer can only view own tickets
- Agent can only view assigned tickets
- Admin can view all tickets
- Admin can assign ticket to agent
- Supervisor can reassign ticket under their team
- Invalid status transition is rejected
- Attachment upload validates file size
- Attachment upload validates file type
- Internal notes are hidden from customers
- API ticket creation works
- Unauthenticated users cannot access protected routes
- Unauthorized users cannot access restricted ticket detail
- SLA due date is calculated when ticket is created

Unit Tests

- Ticket number generator
- SLA due date calculation
- Status transition service

- Overdue detection logic

Minimum:

```
10 tests
```

Better submission:

```
15+ tests
```

Strong submission:

```
20+ tests
```

Testing is required. No tests means the app is just vibes wearing a Laravel hoodie.

README Requirement

The README must include:

- Project overview
- Tech stack
- Installation steps
- Environment setup
- Database migration and seeding
- Queue setup
- Storage link setup
- Test running command
- Seeded user credentials
- Feature summary
- Known limitations
- Developer confession

Example commands:

```
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan storage:link
php artisan queue:work
php artisan test
```

Developer Confession

Add a section in the README called:

```
Developer Confession
```

Answer these:

- What part was hardest?
- What shortcut did you take?
- What would you improve with more time?
- Which part of the code is most cursed but still works?

This is not just for comedy. It helps reviewers understand your judgment, honesty, and technical awareness.

Candidates who say everything is perfect are either lying, confused, or built different in a suspicious way.

Submission

Deadline:

```
One week after the task is given
```

Submission steps:

1. Fork the provided repository
2. Build the project
3. Push your work to your fork

4. Create a pull request to the main repository
5. In the PR description, explain:
    - Your name
    - Tech stack used
    - Main features implemented
    - Any assumptions
    - Any unfinished parts
    - One bug you defeated
    - One cursed thing you are not proud of
    - One thing you would refactor later

Keep the PR description short and useful.

No need to write a Netflix documentary.

Pull Request Checklist

Before submitting, make sure:

- Authentication works
- Role access works
- Ticket CRUD works according to role
- Ticket status workflow is enforced
- Attachments are validated
- Internal notes are protected
- Dashboard uses real data
- API endpoints work
- Tests pass
- README is complete
- No `.env` file is committed
- No `dd()` left behind
- No `dump()` left behind
- No random `console.log()` left behind
- No crime committed during development

Scoring Rubric

Area	Points
Authentication and roles	10
Ticket CRUD and workflow	15
Authorization and policies	15
Database design and relationships	15
Comments, notes, and attachments	10
Dashboard and filtering	10
Notifications and queues	10
API implementation	10
Testing	10
Code quality and structure	15
README and submission quality	5
UI/UX polish	5
Chaos bonus	5

Total:

```
135 points
```

Suggested passing level:

```
85 points
```

Strong submission:

```
105+ points
```

Excellent submission:

```
120+ points
```

Optional Bonus Tasks, Side Quests, and Mildly Suspicious Rituals

These are optional. They can improve the submission, but they do not replace the required features.

Technical Bonus

- Automatically escalate tickets when SLA is violated
- Add realtime ticket updates using Laravel Reverb, Echo, Pusher, or polling
- Add advanced search using Laravel Scout, database full-text search, Meilisearch, or Typesense
- Add Docker setup for app, database, queue worker, and mailpit
- Add OpenAPI documentation for the API
- Add queued export for ticket reports
- Add scheduled command for overdue ticket checks
- Add soft deletes for master data
- Add audit logging using a clean custom implementation or package

UI/UX Bonus

- Add custom empty states
- Add a custom 404 page called `Ticket Lost in the Void`
- Add clear status and priority badges
- Add a `ticket is on fire` indicator for overdue critical tickets
- Add loading states for forms, filters, and tables
- Add mobile-friendly ticket detail layout
- Add a clean notification dropdown
- Add confirmation modal before destructive actions

Documentation Bonus

- Add screenshots to README
- Add API examples
- Add test credentials
- Add architecture notes
- Add database relationship explanation
- Add `Known Limitations` section
- Add `Developer Confession` section

Chaos Bonus

These are optional. They do not replace the main requirements, but they may earn emotional damage points from the reviewer.

- Bake a cake
- Do not commit any crime
- Jadi dukun
- Seed a demo ticket titled `Printer is haunted`
- Seed a demo ticket titled `Production is on fire but calmly`
- Add a `Known Cursed Parts` section in the PR description
- Add a `No dd(), dump(), or console.log left behind` checklist
- Add one funny but professional empty state message
- Add a tiny `touch grass` note after successful test instructions
- Add a harmless `Queue Goblin` name/comment for queued notification logic
- Add a fake but clean `System Health` card only if it is clearly marked as demo data
- Add one warning comment where business logic gets spicy

Important rule:

- Chaos bonus must not break professionalism, security, or clarity.
- Funny is good.
- Leaking internal notes to customers is not funny.
- That is just a bug wearing clown shoes.

Suggested Seed Data

To make the demo easier, seed realistic data.

Suggested categories:

```
Technical Support
Account Issue
Billing
Feature Request
Bug Report
Infrastructure
```

Suggested labels:

```
Urgent
Backend
Frontend
Database
Security
Needs Follow Up
Customer Waiting
```

Suggested priorities:

```
Low
Medium
High
Critical
```

Suggested demo tickets:

```
TCK-2026-000001 - Printer is haunted
TCK-2026-000002 - Cannot login after password reset
TCK-2026-000003 - Production is on fire but calmly
TCK-2026-000004 - Invoice download returns error
TCK-2026-000005 - Need access to admin panel
```

Seed data should help reviewers test the system quickly.

Do not make reviewers manually create everything from zero. That is not an assessment. That is unpaid farming.

Recommended Intermediate-Level Expectations

A candidate should demonstrate:

- Controllers are not bloated
- Business logic is not dumped everywhere
- Policies actually protect the app
- Database relationships are clean
- Validation is consistent
- Tests prove important behavior
- Queues are used properly
- API responses are consistent
- UI is usable
- README is actually helpful
- Code can be reviewed without the reviewer needing spiritual healing

Final Reminder

This challenge is not about adding random features until the app collapses under its own ambition.

The real goal is to build a clean, practical, role-based ticket system with real business rules.

Focus on:

- Clear structure
- Correct authorization
- Clean workflow logic
- Useful tests
- Realistic data modeling
- Good communication in README and PR
- Have fun.
- Do not copy your friend's project.

- Ask your mentor if you get stuck.
- Do not turn the controller into a crime scene.
- Voila.

Last updated 1 minute ago